Semana 18 — Conteúdo de 2 aulas

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Semana 18 — Conteúdo de 2 aulas

- **Aula 29:** Docker: Conceitos e Arquitetura
- **Aula 30:** Práticas com Docker: imagens, contêineres, volumes e redes

Aula 29 — Docker: Conceitos e Arquitetura

Objetivos da Aula

- Compreender a **arquitetura do Docker**.
- Diferenciar **imagem, contêiner e registro**.
- Entender o **Dockerfile** e as camadas.
- Conhecer os principais **comandos**.

O que é Docker?

- Plataforma para **criar, distribuir e executar** contêineres.
- Facilita empacotar aplicações com suas dependências.
- Padroniza o ambiente entre desenvolvimento e produção.
- Base de boa parte dos fluxos de **DevOps**.

Arquitetura do Docker

- **Cliente (CLI):** `docker`.
- **Daemon (`dockerd`):** executa contêineres.
- **Registro:** armazena imagens (ex.: Docker Hub).
- Comunicação cliente ⇔ daemon via **API**.

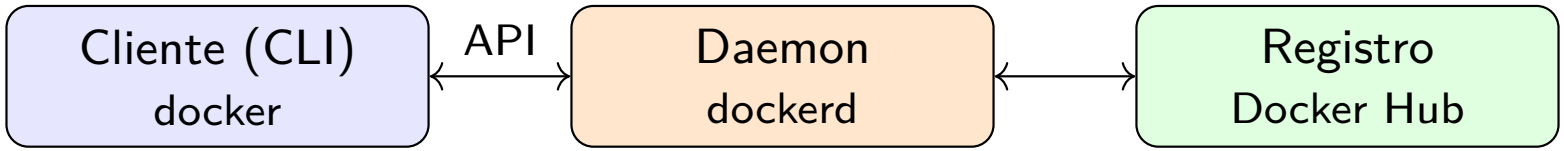


Imagem × Contêiner

- **Imagem:** modelo **somente leitura**, com a aplicação e dependências.
- **Contêiner:** instância **em execução** de uma imagem.
- Analogia: imagem = classe; contêiner = objeto.
- Uma imagem pode gerar **vários** contêineres.

Camadas e Union File System

- A imagem é formada por **camadas** sobrepostas.
- Cada instrução do Dockerfile cria uma **camada**.
- Camadas são **reaproveitadas** (cache) $\rightarrow$ builds rápidos.
- O contêiner adiciona uma camada **gravável** no topo.

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python", "app.py"]
```

- `FROM` define a base; `RUN` executa no build; `CMD` define o comando de execução.

Comandos Essenciais

- `docker build -t minha-app .` — constrói a imagem.
- `docker run minha-app` — executa um contêiner.
- `docker ps` — lista contêineres em execução.
- `docker images` — lista imagens.
- `docker pull/ push` — baixa/envia imagens.

Volumes e Redes (visão)

- **Volumes:** persistência de dados fora do contêiner.
- **Redes:** comunicação entre contêineres (bridge, host, overlay).
- Contêineres são **efêmeros**: dados persistentes vão em volumes.
- Aula 37 detalha volumes e redes.

Registro de Imagens

- **Docker Hub**: registro público padrão.
- Registros **privados** para imagens internas.
- `docker pull` baixa; `docker push` publica.
- Versionamento por **tags** (ex.: `nginx:1.27`).

Tip

Use **imagens base enxutas** (slim/alpine) e fixe **versões** nas tags para builds reprodutíveis.

Resumo da Aula

- Docker tem **cliente, daemon e registro**.
- **Imagem** é o modelo; **contêiner** é a execução.
- Imagens são **camadas**; o Dockerfile as define.
- Comandos: **build, run, ps, images, pull/push**.

Atividade Rápida

- Diferencie **imagem** e **contêiner**.
- O que faz cada linha do **Dockerfile** de exemplo?
- Cite **três comandos** e o que fazem.

Referências

- Docker — *Documentação oficial*.
- Docker Hub.
- Material da aula 33.

Aula 30 — Práticas com Docker: imagens, contêineres, volumes e redes

Objetivos da Aula

- Instalar e verificar o **Docker**.
- Executar contêineres prontos.
- Construir uma **imagem** a partir de um Dockerfile.
- Gerenciar o ciclo de vida de contêineres.

Preparação do Ambiente

- Instalar o **Docker Engine** (ou Docker Desktop).
- Verificar: `docker --version` e `docker run hello-world`.
- No Linux, atenção às permissões (grupo `docker`).
- Confirmar o daemon em execução.

Executando Contêineres Prontos

- `docker run hello-world` — teste básico.
- `docker run -it ubuntu bash` — shell interativo.
- `docker run -d -p 8080:80 nginx` — servidor web em segundo plano.
- Acessar `http://localhost:8080`.

Inspecionando

- `docker ps` — contêineres em execução.
- `docker ps -a` — todos (inclusive parados).
- `docker images` — imagens locais.
- `docker logs <id>` — saída do contêiner.

Construindo uma Imagem

- Criar uma pasta de projeto com `app.py` e `Dockerfile`.
- `docker build -t minha-app:1.0 .`
- `docker run -d -p 8000:8000 minha-app:1.0`
- Verificar o funcionamento.

Gerenciando o Ciclo de Vida

- `docker stop <id>` — parar.
- `docker start <id>` — iniciar.
- `docker rm <id>` — remover contêiner.
- `docker rmi <imagem>` — remover imagem.

Limpeza

- `docker container prune` — remove contêineres parados.
- `docker image prune` — remove imagens sem uso.
- `docker system prune` — limpeza geral (com cuidado).
- Boa prática para não acumular recursos.

Verificação da Prática

- ☐ Docker instalado e `hello-world` executado.
- ☐ Contêineres nginx e ubuntu executados.
- ☐ Imagem própria construída.
- ☐ Contêineres gerenciados (stop/start/rm).
- ☐ Limpeza realizada.

Tip

Mapeamento de portas `-p host:container` é essencial para acessar serviços do contêiner a partir do host.

Resumo da Aula

- Executamos contêineres prontos e **construímos** uma imagem.
- Gerenciamos o **ciclo de vida** com comandos básicos.
- Aprendemos a **limpar** recursos.
- Próxima aula: **volumes e redes** no Docker.

Referências

- Docker — *Get Started*.
- Docker Hub.

Objetivos da Aula

- Compreender a **persistência de dados** com volumes.
- Criar e gerenciar **volumes**.
- Configurar **redes** entre contêineres.
- Orquestrar múltiplos contêineres com o **Compose**.

Por que Persistir Dados?

- Contêineres são **efêmeros**: ao remover, os dados internos se perdem.
- Aplicações reais precisam de **dados persistentes**.
- Solução: **volumes** e **bind mounts**.
- Separa o **dado** do **ciclo de vida** do contêiner.

Volumes no Docker

- `docker volume create meus-dados`
- `docker run -v meus-dados:/var/lib/data ubuntu`
- **Bind mount:** `-v /caminho/host:/caminho/cont`
- Volumes são gerenciados pelo Docker; bind mounts usam pasta do host.

Redes no Docker

- **bridge** (padrão): rede virtual interna.
- **host**: o contêiner usa a rede do host.
- **overlay**: redes entre múltiplos hosts.
- Contêineres na mesma rede se comunicam por **nome**.

Criando uma Rede

- `docker network create minha-rede`
- `docker run --network minha-rede ...`
- Teste de conectividade entre contêineres (`ping`).
- Isolamento e descoberta por **DNS interno**.

Docker Compose

```yaml
services:
  web:
    build: .
    ports: ["8000:8000"]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: exemplo
    volumes:
      - dados:/var/lib/postgresql/data
volumes:
  dados:
```

- Um arquivo descreve **vários** serviços.
- `docker compose up -d` sobe tudo.

Vantagens do Compose

- Ambiente **multi-contêiner** reproduzível.
- Um comando para subir **aplicação + banco**.
- Facilita **desenvolvimento e testes**.
- Base antes de orquestradores maiores (Kubernetes).

Verificação da Prática

- ☐ Volume criado e persistência testada.
- ☐ Rede criada e comunicação validada.
- ☐ `docker-compose.yml` funcionando.
- ☐ Aplicação + banco rodando juntos.

Tip

Nomeie volumes e redes de forma **descritiva** e use o Compose para documentar o ambiente no próprio repositório.

Resumo da Aula

- **Volumes** garantem persistência de dados.
- **Redes** permitem comunicação entre contêineres.
- **Docker Compose** orquestra múltiplos serviços.
- Próxima aula: **atividade sobre contêineres e Docker**.

Referências

- Docker — *Volumes*, *Networking*, *Compose*.